Presence Simulation

JVP Process Interface (Lua Interface) – Help

Recording and time-accurate playback of KNX/EIB telegrams (DPT 1.001, DPT 5.001…5.004) per functional description ch. 4.2.4.11 (Presence Simulation, FP 701 CT IP).

1. Overview

This interface reproduces the presence simulation of a KNX panel as a datapoint model for JUNG Visu Pro: up to 32 freely chosen functions (binary or analog KNX telegrams) can be recorded over any period and then played back on a schedule – typically to simulate occupancy while a home is unoccupied.

The interface is set up as multilingual (German/English). When a new process interface is created, JVP asks for the desired language; folder names, datapoint names and descriptions then appear consistently in that language. The technical *SCRIPTNAME* identifiers (e.g. *RECORDING.RECORDING_LENGTH_DAYS*) do not change – only the visible display names do. The *E:trace()* messages in the trace/log window always stay in English, independent of the chosen language.

2. Datapoint folders

Folder	Contents
RECORDING	Controls/monitors the recording: duration, start/stop, status, telegram counter.
PLAYBACK	Controls/monitors the playback: type (Repeating/Weekday), start delay, start/stop, status.
STATUS	Datapoints shared across folders: selection count, current weekday.
FUNCTIONS	Parent folder containing all created functions (recording objects).
FUNCTIONS.FUNCTION_BINARY	Binary functions (DPT 1.001) – one instance per binary recording object.
FUNCTIONS.FUNCTION_ANALOG	Analog functions (DPT 5.001…5.004) – one instance per analog recording object.

3. Creating a function – input and output

The *FUNCTIONS* folder contains two separate multiple-instance subfolders, one per data format – the one you pick when creating a new instance directly determines the telegram value's type:

Subfolder	Data format	Input (recording)	Output (playback)
FUNCTION_BINARY	DPT 1.001 (1 bit)	VALUE_IN	VALUE_OUT
FUNCTION_ANALOG	DPT 5.001…5.004 (1 byte)	VALUE_IN_A	VALUE_OUT_A

VALUE_IN (or **VALUE_IN_A**) is the input object for recording: connect the KNX group address to be monitored here. **VALUE_OUT** (or **VALUE_OUT_A**) is the output object for playback: connect the group address to be controlled here. Both can point to the same group address (the normal case) or to two different ones (e.g. a status GA for recording, a control GA for playback). Each function also has two matching timestamps: *LASTVALUE_TIME_IN* (last recorded telegram) and *LASTVALUE_TIME_OUT* (last played-back telegram). Per instance there is also *NAME* (plain-text label) and *SELECTED* (included in the simulation).

Note: fields in *FUNCTION_ANALOG* carry an internal *_A* suffix (e.g. *VALUE_IN_A*) because JVP requires unique *SCRIPTNAME* values across the entire interface. This is not relevant for day-to-day use – the script resolves it automatically.

4. Recording

- **RECORDING_START_CMD** / **RECORDING_STOP_CMD** start and stop the recording.
- **RECORDING_LENGTH_DAYS** (1…7) limits the duration as 24-hour periods (not calendar days); recording also stops automatically at 2100 telegrams (MAX_TELEGRAMS).
- Every telegram from a selected function is stored in memory with a precise timestamp (time and weekday) – as with the original device, this is **not persistent**: the buffer is lost when the interface restarts.
- **TELEGRAM_COUNT** shows the number of recorded telegrams; **RECORDING_ACTIVE** indicates whether a recording is in progress.
- Every new recording, or any changed and saved function selection, clears the previous recording (HAS_RECORDING → 0), exactly as required by the functional description.

5. Playback

- **PLAYBACK_TYPE** distinguishes “Repeating” (time of day only, daily cycle) from “Weekday” (time + weekday, weekly cycle; days without a recording are skipped).
- **PLAYBACK_STARTSTOP** starts/stops playback – it corresponds both to the on-screen controls and to the bus-side 1-bit communication object “Start/stop playback”.
- **PLAYBACK_DELAY_HOURS** sets a start delay; progress counts down in **PLAYBACK_DELAY_REMAINING** (seconds) and can be cancelled at any time.
- **PLAYBACK_ACTIVE** indicates whether playback is currently running.
- Telegrams that fall before the actual start time of the current day are – as in the original – picked up in the next cycle.

6. Status & weekday

- **STATUS.SELECTED_COUNT** counts the currently selected functions (binary and analog combined); a 33rd selection is rejected automatically.
- **STATUS.CURRENT_WEEKDAY** (1 = Monday…7 = Sunday) can be written directly, or updated via **STATUS.WEEKDAY_SYNC** from a DPT 10.001 time telegram; without an external source the script advances it automatically each day using the system clock.

7. Installation & commissioning (summary)

- All core files (InterfaceDescription.xml, InterfaceScript.lua, InterfaceScriptCommonLibrary.lua) live in the interface folder – add the interface to the JVP project as usual.

- In the FUNCTIONS folder, create the desired recording objects under FUNCTION_BINARY or FUNCTION_ANALOG depending on data format (32 total) and link VALUE_IN/VALUE_OUT to the real group addresses.
- Build the UI controls (start/stop buttons, selection page, status line) in the visualization based on the datapoints described above.
- Test flow: select functions → start recording → TELEGRAM_COUNT increases on switching → stop recording (HAS_RECORDING = 1) → start playback → linked group addresses receive the recorded values at the exact recorded time.

8. Troubleshooting

The script logs all significant events (init, building the function list, start/stop of recording and playback, every recorded/played-back telegram, rejected operations, unknown datapoints) via *E:trace()* to JVP's trace/log window – if something goes wrong, just include the relevant excerpt. These messages are deliberately English-only, independent of the language chosen for folder/datapoint names.

Note: as with the original device, the recording only lives in memory (not persistent). If HAS_RECORDING/TELEGRAM_COUNT still showed “available” before an interface restart, they are automatically reset to 0 on startup, so that playback cannot appear to start successfully while no telegrams remain.

The full description of every individual datapoint (including its DESCRIPTION attribute) is available directly in InterfaceDescription.xml.